

Графи. Дървета. Обхождания на графи.

1. Дефиниции за краен ориентиран и неориентиран (мулти)граф

1.1. Неориентиран граф

Определение. Неориентиран граф е наредена двойка $G = (V, E)$, където:

- V е непразно крайно множество, чиито елементи се наричат **върхове**;
- $E \subseteq \{X \subseteq V : |X| = 2\}$ е множество от **ребра**, като всяко ребро е двуелементно подмножество на V .

Защо така? Виждаме, че при тази дефиниция ребрата нямат собствена идентичност — реброто е двойката върхове, които свързва. Това означава, че между два върха u и v може да има **най-много едно ребро**, и не са разрешени примки (ребра от вид $\{v, v\}$, защото те биха били едноелементни множества, а изискваме двуелементни). Записът на ребро обикновено е $e_1 = (u, v)$ с кръгли скоби — това е световна конвенция, но смисълът е множество, не наредена двойка.

Стандартни нотации: $n = |V|$, $m = |E|$, винаги $m \leq \binom{n}{2}$.

1.2. Неориентиран мултиграф

Определение. Неориентиран мултиграф е наредена тройка $G = (V, E, f_G)$, където:

- V е непразно крайно множество от върхове;
- E е крайно множество от ребра, при което $V \cap E = \emptyset$;
- $f_G : E \rightarrow \{X \subseteq V : |X| = 2\}$ е **свързваща функция**, която на всяко ребро съпоставя множеството от неговите два края.

Защо нова дефиниция? В мултиграфа ребрата получават **собствена идентичност** като отделни обекти. Така две различни ребра e_1 и e_2 могат да имат едни и същи краища — тоест разрешаваме **паралелни ребра**. Свързващата функция f_G казва кои са краищата на дадено ребро. Сега вече не пишем $e_1 = (u, v)$, а $f_G(e_1) = \{u, v\}$.

Ако позволим още f_G да изобразява в едноелементни множества (т.е. $f_G(e) = \{v\}$), получаваме мултиграф **с примки** (примката е ребро с един и същи връх в двата си края).

При мултиграфи m може да бъде произволно по-голямо от $\binom{n}{2}$ — броят на ребрата вече не е ограничен от броя на върховете.

1.3. Ориентиран граф

Определение. Ориентиран граф (без примки) е наредена двойка $G = (V, E)$, където V е непразно крайно множество от върхове и $E \subseteq V \times V$, като E не съдържа елементи от вида (v, v) .

Защо така? Сега ребрата са **наредени двойки** — има значение кое е началото и кое е краят. Реброто (u, v) води от u към v и това е различно от (v, u) . Графът точно съответства на антирефлексивна релация над V .

Ако махнем условието $(v, v) \notin E$, получаваме **ориентиран граф с възможни примки**, който точно съответства на произволна релация над $V \times V$.

1.4. Ориентиран мултиграф

Определение. Ориентиран мултиграф е наредена тройка $G = (V, E, f_G)$, където:

- V е непразно крайно множество от върхове;
- E е множество от ребра, $V \cap E = \emptyset$;
- $f_G : E \rightarrow V \times V$ е свързваща функция.

Тук разрешаваме както паралелни ребра (различни ребра със същата наредена двойка краища), така и примки ($f_G(e) = (v, v)$).

1.5. Връзка между ориентирани и неориентирани графи

За всеки ориентиран граф G може да дефинираме **съответен неориентиран граф**, получен чрез премахване на ориентацията на ребрата — ако имаме двойка (u, v) и (v, u) , заменяме я с едно неориентирано ребро $\{u, v\}$. Обратно, за неориентиран граф може да дефинираме съответен ориентиран граф, като заместим всяко неориентирано ребро $\{u, v\}$ с **двете** ориентирани ребра (u, v) и (v, u) .

2. Пътища и цикли

2.1. Път и цикъл в неориентиран мултиграф

Определение (път). Нека $G = (V, E, f_G)$ е мултиграф. Път в G е непразна алтернираща редица от върхове и ребра: $p = (u_{i_0}, e_{k_0}, u_{i_1}, e_{k_1}, u_{i_2}, \dots, u_{i_{t-1}}, e_{k_{t-1}}, u_{i_t})$ където $t \geq 0$, всички $u_{i_j} \in V$, всички $e_{k_j} \in E$ и $f_G(e_{k_j}) = \{u_{i_j}, u_{i_{j+1}}\}$ за всяко j .

Върховете u_{i_0} и u_{i_t} се наричат **краища** на пътя, останалите — **вътрешни върхове**. Дължината $|p| = t$ е броят на ребрата.

Пътят е **прост**, ако всички върхове и всички ребра в него са уникални.

Защо алтерниращата редица? В мултиграф между два върха може да има няколко успоредни ребра. Ако пишем само редица от върхове, няма да е ясно през кое ребро „минаваме“. Затова пишем и реброто, и върховете. При обикновен граф между два съседни върха в редицата винаги има най-много едно ребро и затова конвенцията позволява да изпускаме имената на ребрата.

При $t = 0$ имаме тривиален път от един връх с дължина 0 — всеки отделен връх е път сам по себе си.

Определение (цикъл). Цикълът е път, в който $u_{i_0} = u_{i_t}$. Простият цикъл има поне едно ребро и всички елементи (върхове и ребра) са уникални с изключение на това, че началният и крайният връх съвпадат.

Особеност при мултиграфи: Тук са възможни прости цикли с дължина 1 (всяка примка) и с дължина 2 (двойка паралелни ребра между два върха). При обикновените графи това не е възможно — там простите цикли са с дължина поне 3.

2.2. Ориентиран път и ориентиран цикъл

Дефиницията е напълно аналогична, но изискването е $f_G(e_{k_j}) = (u_{i_j}, u_{i_{j+1}})$ — наредена двойка, не множество. Тоест следваме посоката на реброто.

Ключова разлика. В неориентиран граф казваме „път **между** u и v “ — симетрично. В ориентиран казваме „път **от** u **до** v “ — несиметрично. Възможно е да има път от u до v , но да няма път от v до u .

2.3. Свързаност и свързани компоненти

Определение (свързаност в неориентиран граф). Два върха u, v са свързани, ако в G съществува u - v път. Графът G е **свързан**, ако всеки два върха в него са свързани.

Релацията „ u е свързан с v “ е:

- **рефлексивна** — всеки връх е свързан със себе си чрез тривиален път с дължина 0;
- **симетрична** — ако има u - v път, обръщайки го, получаваме v - u път;
- **транзитивна** — това изисква нетривиално доказателство: ако има u - v прост път p и v - w прост път q , искаме да построим u - w прост път. Просто да обединим p и q може да даде нещо, което не е прост път, ако имат други общи върхове освен v .
Поправката: нека b е най-близкият до u връх в q , който се намира и в p . Тогава вземаме подпътя от p от u до b и подпътя от q от b до w . Тяхното обединение е прост път.

Следователно това е **релация на еквивалентност**.

Определение (свързани компоненти). Свързаните компоненти на G са подграфите, индуцирани от класовете на еквивалентност на тази релация. Еквивалентно: те са максималните по включване свързани подграфи.

Защо това определение работи? Всеки клас на еквивалентност съдържа върховете, които взаимно си достижими. Подграфът, индуциран от такъв клас (т.е. съдържащ точно тези върхове и всички ребра между тях), очевидно е свързан и максимален — добавянето на още един връх би изисквало това върхът да е свързан с някой от класа, но тогава той вече щеше да е в класа.

Силна и слаба свързаност (за ориентирани графи). Два върха u, v са **силно свързани** в ориентиран граф, ако има път от u до v и път от v до u . Силно свързаните компоненти са класовете на еквивалентност на тази релация. **Слабо свързаните** компоненти на ориентиран граф са свързаните компоненти на съответния неориентиран граф.

3. Дървета и коренови дървета

3.1. Дърво

Определение (глобална, не-индуктивна). Дърво е граф, който е свързан и ацикличен (т.е. в него няма прости цикли).

Определение (индуктивна, еквивалентна). Множеството от дървета се дефинира така:

- **База.** Всеки граф с един връх и без ребра е дърво.
- **Индуктивна стъпка.** Ако $T = (V, E)$ е дърво, u е връх в T и w е връх, който не е в T , то $T' = (V \cup \{w\}, E \cup \{(u, w)\})$ също е дърво.

Защо тези две дефиниции са еквивалентни? Индуктивната дефиниция показва, че дървото може да се „гради“ от един връх, като всеки път добавяме нов **висящ** връх (т.е. връх от степен 1, окачен на някой от вече съществуващите). Очевидно при такава конструкция графът остава свързан (нов връх се добавя свързан с друг) и ацикличен (не може да се появи цикъл при добавяне на връх само с едно инцидентно ребро). Обратно, ако имаме свързан ацикличен граф, можем да го „разглобим“ постепенно, премахвайки висящи върхове.

Лема (висящи върхове). Всяко дърво с поне 2 върха има поне 2 висящи върха.

Защо? Разглеждаме най-дълъг прост път p в дървото. Краят на p трябва да е висящ — ако имаше други съседни извън пътя, можехме да продължим пътя; ако имаше съсед в пътя, щеше да се появи цикъл. Двата края на p са висящи.

Теорема. В дърво съществува точно един прост път между всеки два върха.

3.2. Теорема: във всяко дърво $m = n - 1$ (т.е. $|V| = |E| + 1$)

Доказателство със структурна индукция съгласно индуктивната дефиниция.

База. Тривиалното дърво има $n = 1$ връх и $m = 0$ ребра. Изпълнено е $0 = 1 - 1$. ✓

Индуктивно предположение. Нека твърдението е вярно за дървото T , т.е. $|E(T)| = |V(T)| - 1$.

Индуктивна стъпка. Дървото T' се получава от T чрез добавяне на нов връх и едно ребро. Тогава: $|V(T')| = |V(T)| + 1$, $|E(T')| = |E(T)| + 1$.

Тогава $|E(T')| = |E(T)| + 1 = (|V(T)| - 1) + 1 = |V(T)| = |V(T')| - 1$.

Доказателството е завършено. ■

3.3. Кореново дърво

Определение. Нека $T = (V, E)$ е (неориентирано) дърво. Избираме произволен връх $r \in V$ и го наричаме **корен**. След избора на корен T става **кореново дърво**.

Изборът на корен еднозначно определя релация на **родителство** върху всяко ребро: за ребро $e = (u, v)$, тъй като в дървото има точно един път от r до всеки връх, единият от u, v е „по-близо“ до корена — той е **родителят**, другият е **детето**.

Стандартно кореновото дърво се рисува „с главата надолу“: коренът най-горе, под него децата, под тях внуците и т.н. Върхът на ниво k означава, че е на разстояние k от корена.

Дефинират се също: **листо** (връх без деца), **вътрешен връх** (не-листо), **дълбочина на връх** (разстоянието до корена), **височина на връх** (максималното разстояние до листо в поддървото), **височина на дървото** (височината на корена), **разклоненост** (максималният брой деца).

3.4. Доказателство, че всяко кореново дърво е дърво и $|V| = |E| + 1$

Това на пръв поглед изглежда тавтологично — нали кореновото дърво се **дефинира** като дърво с избран корен? Но има по-фина гледна точка. Често кореновото дърво се дефинира **индуктивно** без позоваване на не-кореновото:

Индуктивна дефиниция на кореново дърво.

- **База.** Тривиалният граф $(\{u\}, \emptyset)$ е кореново дърво с корен u .
- **Индуктивна стъпка.** Нека $T_1, \dots, T_k$ са коренови дървета с корени $r_1, \dots, r_k$, две по две без общи върхове. Нека r е нов връх. Тогава графът, в който r е свързан с всеки от $r_1, \dots, r_k$ е кореново дърво с корен r .

При тази дефиниция трябва да докажем, че полученият обект е дърво (свързан, ацикличесен граф) и че $|V| = |E| + 1$.

Доказателство (структурна индукция по индуктивната дефиниция).

База. Тривиалният граф $(\{u\}, \emptyset)$ има $|V| = 1$ и $|E| = 0$, очевидно $1 = 0 + 1$. ✓ Той е свързан (тривиално) и ацикличесен (няма ребра). Следователно е дърво.

Индуктивно предположение. Нека за всеки от $T_1, \dots, T_k$ е изпълнено: T_i е дърво и $|V(T_i)| = |E(T_i)| + 1$.

Индуктивна стъпка. Кореновото дърво T има: $|V(T)| = 1 + \sum_{i=1}^k |V(T_i)|$, $|E(T)| = k + \sum_{i=1}^k |E(T_i)|$.

Сумираме индуктивните предположения: $\sum_{i=1}^k |V(T_i)| = \sum_{i=1}^k |E(T_i)| + k$. Тогава:
 $|V(T)| = 1 + \sum |E(T_i)| + k = 1 + (|E(T)| - k) + k = |E(T)| + 1$.

Свързаност: всеки от $T_1, \dots, T_k$ е свързан (по индуктивно предположение); новият корен r е свързан с всеки r_i , следователно с всеки връх в T_i ; значи T е свързан.

Ацикличност: ако в T съществуваше прост цикъл c , той не може изцяло да лежи в кое да е T_i (защото те са ациклични); значи c преминава през r . Но единствените ребра, инцидентни с r , са $(r, r_1), \dots, (r, r_k)$. Цикълът трябва да влиза в r от някой r_i и да излиза към друг r_j . Но тогава между r_i и r_j в обединението $\bigcup T_i$ трябва да съществува път, което е невъзможно — графовете T_i нямат общи върхове, следователно няма как r_i и r_j да са свързани без r . Противоречие.

Следователно T е дърво и $|V(T)| = |E(T)| + 1$. ■

3.5. Покриващо дърво

Определение. Нека $G = (V, E)$ е свързан граф. **Покриващо дърво** на G е дърво $T = (V, E')$, където $E' \subseteq E$ — т.е. T съдържа **всички** върхове на G и **подмножество** от ребрата му.

Теорема (свързаност и наличие на покриващо дърво). Графът G има поне едно покриващо дърво тогава и само тогава, когато е свързан.

Защо? В една посока: ако има покриващо дърво, между всеки два върха в него има път, следователно и в G има път. В другата посока: ако G е свързан, следният прост алгоритъм строи покриващо дърво — докато G има цикъл, избираме произволен цикъл c и произволно ребро $e \in c$ и го изтриваме. Това запазва свързаността (теорема: изтриване на ребро от цикъл на свързан граф не нарушава свързаността) и намалява броя на циклите. Рано или късно получаваме свързан ацикличен граф със същите върхове — покриващо дърво.

4. Обхождане на граф в ширина и в дълбочина

4.1. Обща схема за обхождане

Обхождането на граф означава да посетим всички достижими върхове от даден стартов връх, без да зацикляме. Използва се следната обща схема:

```

Инициализирай visited[v] = False за всеки  $v$ .
 $S := \{1\}$ ; visited[1] := True.
Докато  $S \neq \emptyset$ :
    Извади елемент  $x$  от  $S$ .
    За всеки сосед  $y$  на  $x$ :
        Ако visited[y] = False:
            visited[y] := True
            Добави  $y$  в  $S$ .

```

Различните начини, по които множеството S се реализира, дават различни алгоритми:

- **Опашка (FIFO)** $\rightarrow$ BFS (обхождане в ширина)
- **Стек (LIFO)** или **рекурсия** $\rightarrow$ DFS (обхождане в дълбочина)

4.2. BFS — обхождане в ширина

Алгоритъмът използва **опашка** Q и масив `colour[1..n]` с три възможни стойности:

- **white** — върхът още не е посетен;
- **grey** — върхът е посетен, но още не сме приключили със съседите му (намира се в опашката);
- **black** — върхът е посетен и всички съседни са обработени.

Поддържа се също масивът π — за всеки връх v съхранява родителя му в **дървото на обхождането**, и масивът d — съхранява разстоянието $d[v]$ от стартовия връх до v .

```

BFS(G):
    for  $i := 1$  to  $n$ :
        colour[i] := white
        d[i] :=  $\infty$ 
         $\pi[i] := \text{nil}$ 
    colour[1] := grey
    d[1] := 0
    Enqueue( $Q$ , 1)
    while  $Q \neq \emptyset$ :
         $x := \text{Dequeue}(Q)$ 
        foreach  $y$  in  $\text{adj}[x]$ :
            if colour[y] = white:
                colour[y] := grey
                d[y] := d[x] + 1
                 $\pi[y] := x$ 
                Enqueue( $Q$ ,  $y$ )
        colour[x] := black

```

Защо BFS намира разстоянията? Опашката има FIFO дисциплина — върхове, влезли по-рано, излизат по-рано. Това гарантира, че обхождаме графа **слой по слой**: първо стартовия връх (слой 0), след това всичките му съседни (слой 1), после съседите на съседите, още непосетени (слой 2) и т.н. Понеже всеки връх получава d -стойност от родител си + 1 в момента, когато бива открит, тази стойност точно отговаря на най-късото разстояние от стартовия връх — защото слой k съдържа точно върховете на разстояние k .

Дърво на обхождането на BFS е кореново дърво с корен стартовия връх, в което ребро (x, y) участва тогава и само тогава, когато BFS е открил бял връх y чрез това ребро. Това дърво е покриващо дърво за свързаната компонента, съдържаща стартовия връх. То е „плитко и широко“ — височината му е $\max_v d[v]$.

4.3. DFS — обхождане в дълбочина

Алгоритъмът е **рекурсивен** и неявно използва системния стек.

```
DFS(G):
  for i := 1 to n:
    colour[i] := white
    π[i] := nil
  DFS-VISIT(G, 1)

DFS-VISIT(G, x):
  colour[x] := grey
  foreach y in adj[x]:
    if colour[y] = white:
      π[y] := x
      DFS-VISIT(G, y)
  colour[x] := black
```

Интуитивно описание. Ако си представим графа като лабиринт от стаи и коридори: попадаме в стая, маркираме я като посетена и излизаме през първата неизползвана врата. Ако в новата стая вече сме били — връщаме се. Ако не сме — оставаме в нея и повтаряме. Когато всички врати в дадена стая са използвани — връщаме се една стая назад. Алгоритъмът завършва, когато се върнем в началната стая без неизползвани врати.

DFS изгражда **дървото на DFS**, което също е покриващо дърво на свързаната компонента. За разлика от BFS-дървото, то обикновено е „високо и тясно“. Височината и точната форма зависят от реда на върховете в списъците на съседство.

4.4. Сложност

И двата алгоритъма имат сложност $O(n + m)$ — линейна в размера на графа. Всеки връх се „извежда от опашката/стека“ точно по веднъж, а за всеки извод обработваме всички инцидентни ребра.

5. Ойлерови обхождания на мултиграф

5.1. Дефиниции

Нека G е свързан нетривиален неориентиран мултиграф.

Определение. Ойлеров цикъл в G е цикъл, не непременно прост, който съдържа всяко ребро на G точно веднъж. **Ойлеров път** е път, не непременно прост, който съдържа всяко ребро точно веднъж. G е **Ойлеров**, ако в него има Ойлеров цикъл.

Защо „не непременно прост“? Защото Ойлеровият цикъл/път преминава **през всяко ребро** точно веднъж, но **може да минава многократно през един и същ връх** (когато степента на върха е повече от 2). Затова простостта не е възможна в общия случай.

Тази концепция е свързана със задачата „може ли да се нарисува дадена фигура наведнъж (без вдигане на молива и без повтаряне на линии)?“. Фигурата се моделира с мултиграф — върховете са точките на пресичане, ребрата са отсечките. Може да се нарисува с „един замах“, ако в графа има Ойлеров път.

5.2. Теорема за съществуване на Ойлеров цикъл

Теорема. Свързаният мултиграф G има Ойлеров цикъл тогава и само тогава, когато всеки негов връх е от четна степен.

Доказателство (необходимост — „само тогава“). Допускаме, че G има Ойлеров цикъл c . Ще покажем, че всеки връх е от четна степен.

Разглеждаме произволен връх $u \in V$. Гледаме на c като на кръгова наредба (а не линейна) — върховете и ребрата са разположени върху окръжност. Всяка поява на u в c има точно два съседни елемента — това са две ребра. Тези ребра са инцидентни с u .

Случай без примки в u . На всяка поява на u в c съответстват точно две ребра, инцидентни с u . За различни появи на u , съответните двойки ребра нямат общ елемент (защото c съдържа всяко ребро точно веднъж). Ако u се появява t пъти, общо

получаваме $2t$ ребра, инцидентни с u — но това са **всички** ребра, инцидентни с u (защото c съдържа всяко ребро от графа точно веднъж). Следователно $d(u) = 2t$ — четно число.

Случай с примки. Нека u има q примки. Всяка примка e_i в c се появява като подредица u, e_i, u (два пъти u с примката между тях). Всяка такава тривиална „обиколка“ допринася 2 към степента на u . Останалите $t - q$ появи на u са обхождания между u и негови съседи (не самия u), за всяка от които u има две съседни ребра в c . Сумирайки: $d(u) = 2q + 2(t - q) = 2t$ — пак четно. ■

Доказателство (достатъчност — „тогава“). Допускаме, че G е свързан и всеки връх е от четна степен. Ще покажем съществуването на Ойлеров цикъл **конструктивно** чрез алгоритъм. Това е алгоритъмът на **Hierholzer** (XIX век).

Идеята. Започваме от произволен връх a и „вървим“ — на всяка стъпка преминаваме през произволно **още неизползвано** ребро, инцидентно с текущия връх. Това не може да блокира преди да се върнем в a — защото при всяко влизане в произволен връх $v \neq a$, ние сме „използвали“ нечетен брой ребра, инцидентни с v , а $d(v)$ е четно, така че винаги има още неизползвано ребро да излезем. Само в a е възможно да блокираме — там ще се случи, когато сме „използвали“ всички инцидентни ребра.

Така получаваме цикъл c , но е възможно той да не съдържа всички ребра. Тогава, тъй като G е свързан, в c има връх u , който е инцидентен с поне едно неизползвано ребро. Започвайки от u , по същия начин конструираме нов цикъл s (от неизползваните ребра, отново всеки връх „вижда“ четен брой такива, защото степените в подграфа на неизползваните ребра също са четни — четно минус четно е четно). **Вграждаме** s в c на мястото на u : вместо да преминем през u веднъж, „завиваме“ в s и след пълна обиколка се връщаме в u , продължавайки c .

Повтаряме, докато всички ребра са използвани. Резултатът е Ойлеров цикъл. ■

5.3. Теорема за съществуване на Ойлеров път (който не е цикъл)

Теорема. Свързаният мултиграф G има Ойлеров път, който **не** е цикъл, тогава и само тогава, когато точно два върха са от нечетна степен.

Доказателство. „Само тогава“. Нека G има Ойлеров u - v път p с $u \neq v$. Добавяме ново ребро e между u и v , получавайки мултиграф G_1 . Тогава p заедно с e образува Ойлеров цикъл в G_1 . От Теорема 23, всички върхове в G_1 са от четна степен. Но $d_{G_1}(u) = d_G(u) + 1$ и $d_{G_1}(v) = d_G(v) + 1$, докато за всеки друг връх $d_{G_1}(x) = d_G(x)$. Следва: $d_G(u), d_G(v)$ са нечетни, всички останали — четни.

„Тогава“. Нека u, v са единствените два върха от нечетна степен. Добавяме ребро e между тях, получавайки G_2 , в който всички степени са четни. По Теорема 23, в G_2 има Ойлеров цикъл c . Изтривайки e от c , получаваме Ойлеров u - v път в G . ■

Забележка за броя на върховете от нечетна степен. Като следствие от лемата за сумата на степените ($\sum d(v) = 2m$ — четно), броят на върховете от нечетна степен винаги е четен. Така че възможностите са: 0 (има Ойлеров цикъл), 2 (има Ойлеров път, който не е цикъл), 4, 6, ... (няма нито цикъл, нито път).

ПРИМЕРНИ ЗАДАЧИ

Задача 1. Построяване на покриващо дърво (BFS или DFS)

Постановка. Даден е следният свързан неориентиран граф $G = (V, E)$ с върхове $V = \{1, 2, 3, 4, 5, 6, 7\}$ и списъци на съседство:

```
1: 2, 4
2: 1, 4, 5, 7
3: 5, 7
4: 1, 2, 6, 7
5: 2, 3, 7
6: 4, 7
7: 2, 3, 4, 5, 6
```

Да се построи покриващо дърво на G чрез BFS, започвайки от връх 1.

Решение. Изпълняваме BFS от връх 1.

Стъпка 0 (инициализация): Всички върхове са бели, $d[1] = 0$, опашка $Q = [1]$, връх 1 е сив.

Стъпка 1 (изваждаме 1): Съседите му в реда от списъка са 2, 4. И двата са бели:

- $\pi[2] := 1, d[2] := 1$, връх 2 става сив, влиза в опашката.
- $\pi[4] := 1, d[4] := 1$, връх 4 става сив, влиза в опашката.

В дървото влизат ребрата $(1, 2)$ и $(1, 4)$. Опашка: $Q = [2, 4]$.

Стъпка 2 (изваждаме 2): Съседите са 1, 4, 5, 7.

- 1 е сив (всъщност черен сега) — пропуска се.

- 4 е сив — пропуска се.
- 5 е бял: $\pi[5] := 2, d[5] := 2$, в опашката.
- 7 е бял: $\pi[7] := 2, d[7] := 2$, в опашката.

В дървото влизат (2, 5) и (2, 7). Опашка: $Q = [4, 5, 7]$.

Стъпка 3 (изваждаме 4): Съседите са 1, 2, 6, 7.

- 1, 2 — не бели.
- 6 е бял: $\pi[6] := 4, d[6] := 2$, в опашката.
- 7 — вече е сив (стана сив при обработка на връх 2), пропуска се.

В дървото влиза (4, 6). Опашка: $Q = [5, 7, 6]$.

Стъпка 4 (изваждаме 5): Съседите са 2, 3, 7.

- 2, 7 — не бели.
- 3 е бял: $\pi[3] := 5, d[3] := 3$, в опашката.

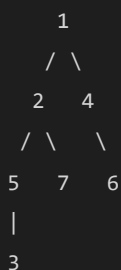
В дървото влиза (5, 3). Опашка: $Q = [7, 6, 3]$.

Стъпки 5, 6, 7 (изваждаме 7, 6, 3 *порежд*): Всички съседни на тези върхове вече не са бели — нищо ново не се добавя.

Резултат: Покриващото дърво T има ребра: $E(T) = \{(1, 2), (1, 4), (2, 5), (2, 7), (4, 6), (5, 3)\}$

То има $|V(T)| = 7$ върха и $|E(T)| = 6$ ребра — съответства на формулата $|E| = |V| - 1$.
✓

Дървото като кореново с корен 1:



Защо го получихме така? BFS обработва върховете по нарастваща дълбочина: първо тези на разстояние 1 от старта (нивото на 2 и 4), после на разстояние 2 (5, 7, 6), после на разстояние 3 (3). Реалното разстояние от 1 до всеки връх в графа точно съвпада с

разстоянието в дървото на BFS — това е ключово свойство на BFS, което DFS **не** притежава.

Задача 2. Ойлеров цикъл или Ойлеров път

Постановка. Даден е следният неориентиран мултиграф G с върхове $V = \{a, b, c, d, e\}$ и ребра: $E = \{(a, b), (a, c), (a, d), (b, c), (b, d), (c, d), (c, e), (d, e)\}$

Да се построи Ойлеров цикъл или Ойлеров път, или да се докаже, че такъв не съществува.

Решение. Първо проверяваме степените:

- $d(a) = 3$ (съседни: b, c, d) — нечетна
- $d(b) = 3$ (съседни: a, c, d) — нечетна
- $d(c) = 4$ (съседни: a, b, d, e) — четна
- $d(d) = 4$ (съседни: a, b, c, e) — четна
- $d(e) = 2$ (съседни: c, d) — четна

Имаме **точно два** върха от нечетна степен (a и b). Графът е свързан (лесно се проверява). Следователно:

- Ойлеров цикъл **няма** (Теорема 23 изисква всички степени да са четни — нарушено).
- Ойлеров a - b път **има** (Теорема 24).

Построение на Ойлеровия път (използваме идеята на Hierholzer, започвайки от a , тъй като той е от нечетна степен):

Започваме от a . „Вървим“, избирайки произволни неизползвани ребра:

- $a \rightarrow b$ чрез (a, b) . Текущо: a, b .
- $b \rightarrow c$ чрез (b, c) . Текущо: a, b, c .
- $c \rightarrow a$ чрез (a, c) . Текущо: a, b, c, a .
- $a \rightarrow d$ чрез (a, d) . Текущо: a, b, c, a, d .
- $d \rightarrow b$ чрез (b, d) . Текущо: a, b, c, a, d, b . Тук от b нямаме повече неизползвани ребра.

Получихме път от a до b с дължина 5, но в графа има 8 ребра. Остават неизползвани: $(c, d), (c, e), (d, e)$. Те образуват триъгълник c - d - e .

Намираме връх по пътя, който е инцидентен с неизползвано ребро. Например връх c — от него тръгва (c, d) .

Започваме от c нов цикъл (тук той се затваря в c — тъй като в подграфа на оставащите ребра всички степени са четни):

- $c \rightarrow d$ чрез (c, d) .
- $d \rightarrow e$ чрез (d, e) .
- $e \rightarrow c$ чрез (c, e) . Затваря се в c . Малък цикъл: c, d, e, c .

Вграждаме този цикъл в пътя на мястото на първата поява на c :

Първоначално: $a, b, \boxed{c}, a, d, b$. След вграждане в c : $a, b, [c, d, e, c], a, d, b$.

Сглобено: $a, b, c, d, e, c, a, d, b$.

Проверка: ребра в реда им — $(a, b), (b, c), (c, d), (d, e), (e, c), (c, a), (a, d), (d, b)$. Това са точно 8 ребра, всяко използвано точно веднъж. ✓

Резултат: Ойлеров a - b път: $a, b, c, d, e, c, a, d, b$.

Защо алгоритъмът работи? Започвайки от връх от нечетна степен, „вървим“, докато не блокираме. Тъй като при влизане във вътрешен връх използваме нечетен брой ребра, но степента е четна — винаги има как да излезем. Можем да блокираме само във връх от нечетна степен. Имаме два такива — a (стартът) и b . Затова блокираме в b . Полученият път използва някои ребра. За останалите ребра, всеки връх в индуцирания от тях подграф има четна степен — затова те се разбиват на цикли, които вграждаме в първоначалния път.

Задача 3. Разбиване на ребрата на минимален брой пътища без общи ребра

Постановка. Даден е свързан неориентиран граф $G = (V, E)$ с върхове $V = \{1, 2, 3, 4, 5, 6\}$ и ребра $E = \{(1, 2), (1, 3), (2, 3), (2, 4), (3, 4), (4, 5), (4, 6), (5, 6)\}$.

Да се разбие E на **минимален** брой пътища, никои два от които нямат общо ребро.

Теоретична основа. Тази задача е тясно свързана с Ойлеровата теория.

Ключова теорема. Нека G е свързан неориентиран граф (или мултиграф) с $2k$ върха от нечетна степен. Тогава:

- Ако $k = 0$, G е Ойлеров и E е едно цяло — един Ойлеров цикъл, който може да се разглежда като 1 път.
- Ако $k \geq 1$, минималният брой пътища, на които можем да разбием E , без два от тях да имат общо ребро, е k .

Защо именно k ? Всеки път в разбиването има най-много **два края** (когато не е цикъл). Във всеки край, пътът „консумира“ нечетен брой ребра, инцидентни с този връх. В останалите върхове (вътрешните по пътя или непосетените от него) — четен брой.

Сумирано по всички пътища: за всеки връх v , броят на пътищата, които „завършват“ в v (т.е. v е техни край), трябва да има същата четност като $d(v)$. Това следва от факта, че всеки път, при който v не е край, използва четен брой ребра, инцидентни с v (влиза и излиза); всеки път, при който v е край, използва нечетен брой.

Ако v е от нечетна степен, поне един път трябва да завършва в v . Понеже всеки път има 2 края, общо има $2(\text{брой пътища})$ „слотове за край“. За да обслужим всичките $2k$ върха от нечетна степен, трябва **поне k пъти**.

Конструктивно: тази долна граница k е достижима. Доказателство: добавяме „виртуални“ ребра, които вдвояват двойките нечетни върхове помежду им — добавяме k нови ребра. Получаваме граф (или мултиграф), в който всеки връх е от четна степен и графът е Ойлеров. Намираме Ойлеров цикъл в новия граф. Изтриваме виртуалните ребра от него — те разделят цикъла на точно k части, всяка от които е път в оригиналния граф. Тези пътища нямат общи ребра (защото произхождат от един и същ Ойлеров цикъл, в който всяко ребро е използвано веднъж). ■

Решение на конкретната задача. Изчисляваме степените:

- $d(1) = 2$ (съседни: 2, 3) — четна
- $d(2) = 3$ (съседни: 1, 3, 4) — нечетна
- $d(3) = 3$ (съседни: 1, 2, 4) — нечетна
- $d(4) = 4$ (съседни: 2, 3, 5, 6) — четна
- $d(5) = 2$ (съседни: 4, 6) — четна
- $d(6) = 2$ (съседни: 4, 5) — четна

Имаме **2 върха от нечетна степен**: 2 и 3. Следователно $k = 1$ и минималният брой пътища е **1**.

Тоест, в G съществува Ойлеров 2-3 път, и той сам по себе си е „разбиването“ на E на минимален брой пътища.

Намираме го с Hierholzer, започвайки от 2:

- $2 \rightarrow 1$ чрез $(1, 2)$.
- $1 \rightarrow 3$ чрез $(1, 3)$.
- $3 \rightarrow 4$ чрез $(3, 4)$.
- $4 \rightarrow 5$ чрез $(4, 5)$.
- $5 \rightarrow 6$ чрез $(5, 6)$.
- $6 \rightarrow 4$ чрез $(4, 6)$.
- $4 \rightarrow 2$ чрез $(2, 4)$. Текущо: 2, 1, 3, 4, 5, 6, 4, 2. Тук блокираме в 2 — но не сме обходили реброто $(2, 3)$.

Хм — получихме цикъл 2, 1, 3, 4, 5, 6, 4, 2 от 7 ребра, но има 8 ребра в графа. Остава $(2, 3)$. Това е път, не цикъл — единственото неизползвано ребро.

Намираме връх в текущия маршрут, инцидентен с $(2, 3)$. Това е връх 2 или 3. Тръгваме от 3 в подграфа на неизползваните ребра — там степените са: $d(2) = 1, d(3) = 1$, останалите 0. Това не е Ойлеров цикъл, а Ойлеров път.

Това означава, че началната ни „разходка“ не започна добре. Нека пробваме отново, но избирайки по-разумно първото ребро от 2:

Започваме от 2:

- $2 \rightarrow 3$ чрез $(2, 3)$.
- $3 \rightarrow 1$ чрез $(1, 3)$.
- $1 \rightarrow 2$ чрез $(1, 2)$.
- $2 \rightarrow 4$ чрез $(2, 4)$.
- $4 \rightarrow 3$ чрез $(3, 4)$. Текущо: 2, 3, 1, 2, 4, 3. Блокирахме в 3.

Имаме път 2, 3, 1, 2, 4, 3 с 5 ребра. Останали неизползвани: $(4, 5), (4, 6), (5, 6)$ — триъгълник около връх 4.

Намираме връх по пътя, инцидентен с неизползвано ребро. Това е 4. От 4:

- $4 \rightarrow 5, 5 \rightarrow 6, 6 \rightarrow 4$ — цикъл 4, 5, 6, 4.

Вграждаме в първоначалния път на мястото на 4:

Първоначално: 2, 3, 1, 2, 4, 3. Вграждаме: 2, 3, 1, 2, [4, 5, 6, 4], 3. Сглобено: 2, 3, 1, 2, 4, 5, 6, 4, 3.

Проверка на ребрата: $(2, 3), (1, 3), (1, 2), (2, 4), (4, 5), (5, 6), (4, 6), (3, 4)$ — 8 ребра, всяко използвано точно веднъж. ✓

Резултат: Минималният брой пътища е 1, а разбиването е: $P_1 : 2 \rightarrow 3 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 4 \rightarrow 3$.

Бележка. Ако графът имаше повече върхове от нечетна степен (например 4), отговорът щеше да е $k = 2$, и щяхме да приложим описания трик с виртуални ребра — добавяме две виртуални ребра, вдвоявайки 4-те върха от нечетна степен по двойки, намираме Ойлеров цикъл в така допълнения граф и след това „разрязваме“ цикъла на местата на виртуалните ребра, получавайки точно 2 пътя.

Кратък обобщителен лист

Понятие	Ключово свойство
Свързан граф	Има път между всеки два върха
Дърво	Свързан + ацикличен; $\Leftrightarrow m = n - 1$ за свързан граф
Покриващо дърво	Подграф със същите върхове, който е дърво; съществува $\Leftrightarrow$ графът е свързан
BFS	FIFO опашка; намира най-къси пътища (брой ребра) от стартовия връх
DFS	Рекурсия / LIFO стек; полезно за откриване на цикли, силно свързани компоненти, топологична сортировка
Ойлеров цикъл	Всички степени четни (плюс свързаност)
Ойлеров път (не цикъл)	Точно 2 върха от нечетна степен
Разбиване на ребрата на минимален брой пътища	При $2k$ върха от нечетна степен — минимум k пътя (при $k \geq 1$); при $k = 0$ — 1 Ойлеров цикъл